Sorting Benchmark Lab

CS314 Divide and Conquer - Mini Project Report

1. Problem

Which sorting algorithm should we use, and how does the answer change with the input size and the input order? This project compares three algorithms on sorted, reversed and random data for n = 10^3 up to 10^6: **insertion sort**, **merge sort** (a divide-and-conquer algorithm) and **Python's built-in sorted()**.

2. Method

All algorithms are written in Python and return a sorted copy (`sorts.py`). Each result is checked against `sorted()` and the code is covered by 14 unit tests (`test_sorts.py`: empty and single inputs, duplicates, negatives, stability, and the comparison/shift counts of insertion sort). Time is measured with `time.perf_counter()`, taking the best of 3 runs for n <= 10,000 and a single run above that. Random data uses a fixed seed (42). Because insertion sort is quadratic, it was measured only up to n = 10,000; larger sizes would take hours, so they are extrapolated instead.

3. Expected complexity

Algorithm	Best case	Average	Worst case	Extra space
Insertion sort	O(n) sorted	O(n^2)	O(n^2) reversed	O(1)
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Python sorted() (Timsort)	O(n)	O(n log n)	O(n log n)	O(n)

Merge sort follows the divide-and-conquer pattern: split into halves, sort each half recursively, merge. Its recurrence T(n) = 2T(n/2) + O(n) solves to O(n log n) by the Master theorem. Timsort detects already-ordered runs, which is why it reaches O(n) on sorted or reversed input.

4. Results

n = 10,000

Input	Insertion	Merge sort	sorted()
sorted	1.64 ms	30.1 ms	0.09 ms
reversed	3.74 s	14.4 ms	0.08 ms
random	1.70 s	18.2 ms	1.23 ms

n = 1,000,000

Input	Insertion	Merge sort	sorted()
sorted	not run (est. hours)	2.05 s	14.3 ms
reversed	not run (est. hours)	3.02 s	22.0 ms
random	not run (est. hours)	4.46 s	364.9 ms

5. Figures

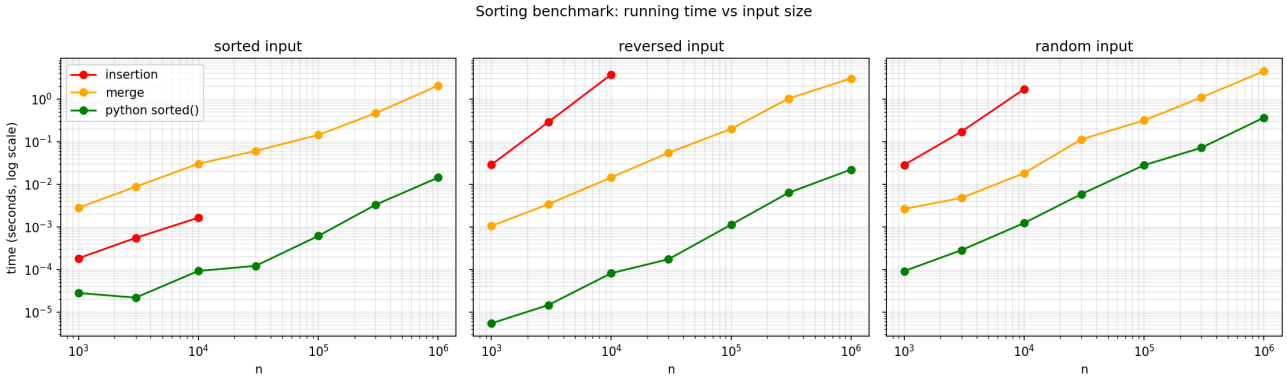


Figure 1: running time vs n on log-log axes. A steeper line means faster growth.

6. Analysis

Insertion sort is quadratic. On reversed input, growing n by 10x (1,000 to 10,000) multiplied the time by 131x, close to the 100x predicted by $O(n^2)$ (the extra factor is probably from cache and memory effects). Extrapolating, $n = 10^6$ would take about 10 hours, so insertion sort is unusable at that scale.

But insertion sort wins on sorted input. At n = 10,000 it took 1.64 ms against 30.1 ms for merge sort, because it does one comparison per element and never shifts, while merge sort still splits and copies the whole list.

Merge sort grows like n log n. Going from n = 100,000 to 1,000,000 on random data multiplied the time by 14.2x, which matches the expected ~12x for n log n and is far below 100x. Its time depends only mildly on input order.

Python's sorted() is the fastest. On 10^6 random numbers it needs 364.9 ms versus 4.46 s for our merge sort (about 12x faster). Part of this gap is implementation (sorted() is written in C, ours runs in the Python interpreter), so the comparison shows practical speed, not only algorithmic difference. On sorted or reversed input sorted() takes only 14.3 ms and 22.0 ms, because Timsort finds the existing runs and does O(n) work.

7. Conclusion

The experiments agree with the theory. Use insertion sort only for small or nearly sorted data; use an O(n log n) divide-and-conquer sort for general large inputs; and in practice use the library sort. Timsort combines both ideas, merging runs and using insertion sort on small pieces, which is why it is good in every case tested. A natural follow-up is the Hybrid Sort project: switch merge sort to insertion sort below a cutoff size and find the best cutoff.

8. Reproducing

`pip install matplotlib`, then `python -m unittest -v` and `python benchmark.py`. Raw data: `results.csv`.